



**SENAC NAÇÕES UNIDAS**  
Oscilações e Ondas

## **Relatório Técnico: Analisador SpectraWave**

Processamento Digital de Sinais, Decomposição Espectral e Aplicação  
Computacional da Transformada de Fourier

Gabriela Lima Gomes da Silva  
Giulia Gabriela Silva de Freitas  
Karoline Gonçalves Ferraz de Campos  
Matheus Yoshio Salies Taqueushi  
Pablo dos Santos Vargas

**19 de maio de 2026**

# Sumário

|          |                                                                         |           |
|----------|-------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introdução ao SpectraWave</b>                                        | <b>2</b>  |
| <b>2</b> | <b>Fundamentação Teórica</b>                                            | <b>3</b>  |
| 2.1      | A Série de Fourier: O Universo Cíclico e Contínuo . . . . .             | 3         |
| 2.2      | A Transformada Discreta de Fourier (DFT): O Enfoque Principal . . . . . | 3         |
| 2.2.1    | Dissecando a Equação da DFT através de Euler . . . . .                  | 4         |
| 2.2.2    | A Intuição Geométrica do Enrolamento e da Ressonância . . . . .         | 4         |
| 2.2.3    | Extração Algébrica de Magnitude e Fase . . . . .                        | 4         |
| <b>3</b> | <b>Guia de Utilização da Interface Visual</b>                           | <b>6</b>  |
| <b>4</b> | <b>Arquitetura do Código-Fonte e Implementação Direta</b>               | <b>7</b>  |
| 4.1      | Interface e Renderização Gráfica: <code>index.html</code> . . . . .     | 7         |
| 4.2      | O Algoritmo de Alta Performance: <code>fft.js</code> . . . . .          | 7         |
| 4.3      | O Cérebro do Sistema: <code>analyzer.js</code> . . . . .                | 8         |
| <b>5</b> | <b>Análise Prática: Estudo de Caso com Audio_Teste.ogg</b>              | <b>10</b> |
| 5.1      | Identificação do Arquivo . . . . .                                      | 10        |
| 5.2      | Análise Visual dos Gráficos de Domínio . . . . .                        | 10        |
| 5.3      | Dissecação das Métricas e Parâmetros Obtidos . . . . .                  | 11        |
| 5.4      | A Equação da Transformada Discreta de Fourier e o Bin 167 . . . . .     | 12        |
| 5.5      | A Série de Fourier Expandida do Sinal . . . . .                         | 14        |
| <b>6</b> | <b>Conclusão</b>                                                        | <b>16</b> |

# 1 Introdução ao SpectraWave

O projeto **SpectraWave** foi desenvolvido no escopo da disciplina de Oscilações e Ondas como uma ferramenta analítica computacional para ambientes web. O propósito do sistema é receber um sinal acústico no domínio do tempo, isolar os intervalos de máxima densidade energética e realizar o mapeamento algorítmico de seus componentes espectrais.

A aplicação foi projetada utilizando métodos de implementação nativa, abdicando de bibliotecas matemáticas de alto nível. O sistema aplica algoritmos de Processamento Digital de Sinais (DSP) baseados em álgebra linear, permitindo a análise de fenômenos ondulatórios e a extração de métricas físicas diretamente através do processamento computacional em tempo real.

## 2 Fundamentação Teórica

Para entender como o SpectraWave funciona, imagine uma onda sonora como uma receita complexa de um bolo. Se você comer o bolo inteiro de uma vez, sentirá um sabor único e complexo (este é o som no **Domínio do Tempo**). A análise de Fourier funciona como um processo químico capaz de separar esse bolo exatamente em suas colheres originais de açúcar, farinha e ovos isolados (estas são as frequências puras no **Domínio da Frequência**).

### 2.1 A Série de Fourier: O Universo Cíclico e Contínuo

A Série Trigonométrica de Fourier estabelece que *qualquer* onda ou oscilação que se repita perfeitamente ao longo do tempo (um sinal periódico) pode ser construída somando-se uma quantidade infinita de ondas senoidais e cossenoidais puras de frequências múltiplas. A sua equação analítica elementar é dada por:

$$f(t) = \frac{a_0}{2} + \sum_{n=1}^{\infty} [a_n \cos(2\pi n f_0 t) + b_n \sin(2\pi n f_0 t)] \quad (1)$$

Para compreender detalhadamente o que ocorre nessa modelagem, dissecamos cada termo a seguir:

- $f(t)$ : É o valor instantâneo da amplitude da onda real (pressão sonora) medida no exato momento de tempo  $t$ .
- $\frac{a_0}{2}$ : Chamado de Componente DC (ou Corrente Contínua). Representa a média matemática de todos os pontos da onda. Fisicamente, indica se a onda possui um deslocamento vertical constante (um desvio de eixo) ou se ela oscila perfeitamente simétrica em torno do zero.
- $f_0$ : A Frequência Fundamental. É a taxa de oscilação mais baixa e longa que define o compasso principal de repetição da onda. Ela dita a "nota musical" percebida pelo ouvido humano.
- $n$ : O número inteiro indicador da harmônica. Se a frequência fundamental  $f_0$  for 100 Hz, a harmônica  $n = 2$  será uma onda de 200 Hz, a harmônica  $n = 3$  será de 300 Hz, e assim sucessivamente.
- $a_n$  e  $b_n$ : São os **coeficientes espectrais contínuos**. Eles funcionam como controles de volume individuais, informando a intensidade exata que cada componente de cosseno ( $a_n$ ) e de seno ( $b_n$ ) precisa ter para que a soma de todos resulte na onda complexa original.

### 2.2 A Transformada Discreta de Fourier (DFT): O Enfoque Principal

A Série de Fourier contínua descrita acima é uma ferramenta idealizada para sinais infinitos. Contudo, computadores não operam com o infinito e placas de áudio capturam apenas uma sequência finita de pontos discretos (amostras). Para resolver esse cenário do mundo real, migramos para a **Transformada Discreta de Fourier (DFT)**.

O objetivo matemático da DFT é converter um vetor finito de  $N$  amostras temporais, denotado por  $x[n]$ , em um vetor de amplitudes complexas de frequência, denotado por  $X[k]$ . A equação matemática que governa este processo é:

$$X[k] = \sum_{n=0}^{N-1} x[n] \cdot e^{-i \frac{2\pi kn}{N}} \quad (2)$$

### 2.2.1 Dissecando a Equação da DFT através de Euler

Para um estudante iniciante, a presença da base exponencial complexa elevada a um número imaginário ( $-i$ ) pode parecer intimidadora. O segredo para desmistificar essa equação reside na famosa **Identidade de Euler**, que mapeia funções exponenciais complexas diretamente no círculo trigonométrico:

$$e^{-i\theta} = \cos(\theta) - i \sin(\theta) \quad (3)$$

Substituindo a Identidade de Euler na fórmula padrão da DFT, conseguimos separar a equação matemática de forma transparente em duas componentes somatórias simples:

$$X[k] = \underbrace{\sum_{n=0}^{N-1} x[n] \cos\left(\frac{2\pi kn}{N}\right)}_{\text{Parte Real (a)}} - i \underbrace{\sum_{n=0}^{N-1} x[n] \sin\left(\frac{2\pi kn}{N}\right)}_{\text{Parte Imaginária (b)}} \quad (4)$$

### 2.2.2 A Intuição Geométrica do Enrolamento e da Ressonância

O significado físico dessa equação pode ser visualizado através de uma analogia geométrica: imagine que você pega a linha contínua do gráfico da sua onda sonora e a "enrola" ao redor de um círculo com velocidade de rotação ajustável. A variável  $k$  representa essa velocidade de rotação (o bin de teste de frequência).

- Se a frequência real presente no áudio **não coincidir** com a velocidade de teste  $k$ , a onda será distribuída de forma homogênea e uniforme por todos os quadrantes do círculo. Suas cristas e vales se anularão mutuamente, fazendo com que o somatório resulte em um valor nulo ou próximo de zero.
- Se a frequência presente no áudio **coincidir perfeitamente** com a velocidade de rotação  $k$ , ocorrerá o fenômeno da **ressonância matemática**. Todas as cristas da onda se acumularão concentradas do mesmo lado do círculo, deslocando o centro de massa geométrico para longe da origem. Esse acúmulo gera um vetor de alta magnitude.

O somatório associado ao cos extrai o quanto essa projeção de massa puxa o círculo no eixo horizontal (eixo  $X$ , gerando a **Parte Real a**). O somatório associado ao sin extrai o quanto essa força puxa no eixo vertical (eixo  $Y$ , gerando a **Parte Imaginária b**).

### 2.2.3 Extração Algébrica de Magnitude e Fase

Ao calcular os dois somatórios para um bin  $k$ , o sistema obtém uma coordenada no plano complexo na forma cartesiana:  $X[k] = a + bi$ . A partir deste par de coordenadas puras, o SpectraWave extrai as propriedades físicas reais percebidas no fenômeno ondulatório:

1. **Magnitude** ( $|X[k]|$ ): Representa a amplitude absoluta ou "força" daquela frequência específica. Matematicamente, calcula-se a distância linear da coordenada até a origem através do Teorema de Pitágoras:

$$|X[k]| = \sqrt{a^2 + b^2} \quad (5)$$

2. **Fase ( $\phi_k$ ):** Representa o deslocamento no tempo (atraso angular) da onda ao iniciar. Obtém-se esse parâmetro em radianos via trigonometria cartesiana:

$$\phi_k = \arctan\left(\frac{b}{a}\right) \quad (6)$$

Essa operação converte as coordenadas retangulares para a elegante **Notação Polar de Fourier**:  $X[k] = |X|e^{i\phi}$ , que é exatamente o formato matemático exibido na interface do SpectraWave após o processamento.

### 3 Guia de Utilização da Interface Visual

A interface do SpectraWave v6.0 foi desenhada com uma estética minimalista de terminal escuro de alta legibilidade. O fluxo de operação do sistema compreende os seguintes passos sequenciais:

1. **Acomodação do Arquivo:** O usuário interage com a região tracejada denominada **dropZone**. É possível arrastar um arquivo de áudio (nos formatos **.ogg**, **.wav**, **.mp3**) ou clicar sobre a área para abrir o seletor nativo do sistema operacional.
2. **Processamento Automatizado:** Ao carregar o arquivo, o sistema oculta a área de transferência e aciona o indicador visual de carregamento. Neste instante, ocorrem em plano de fundo a decodificação dos buffers binários pelo **AudioContext**, a unificação dos canais estéreo em sinal monofônico, a varredura de potência RMS e a execução da FFT de forma transparente ao usuário.
3. **Leitura Espectral:** Finalizados os cálculos, o painel de resultados se torna visível, apresentando de forma estruturada os gráficos de domínio temporal e de densidade de potência, seguidos pelo quadro de metadados escalares e pelo relatório matemático avançado com as fórmulas renderizadas em tempo real.

## 4 Arquitetura do Código-Fonte e Implementação Direta

Para comprovar a viabilidade técnica de processar DSP sem o auxílio de bibliotecas externas, dividimos o sistema em três módulos estruturais complementares.

### 4.1 Interface e Renderização Gráfica: `index.html`

O arquivo de interface gerencia a estrutura visual da árvore DOM e incorpora as folhas de estilo CSS baseadas em variáveis globais. O ponto crítico de engenharia neste arquivo é a importação e configuração das dependências do **KaTeX** via links hipertextuais seguros. O script injeta uma chamada para `window.renderMathInElement`, permitindo a varredura e conversão instantânea de delimitadores de string do tipo `$$` em equações matemáticas vetoriais nítidas.

### 4.2 O Algoritmo de Alta Performance: `fft.js`

Como visto na fundamentação teórica, calcular a DFT pela equação bruta demanda uma complexidade de tempo de  $O(N^2)$ . Para tornar a aplicação viável no navegador, o arquivo `fft.js` implementa a estratégia de divisão e conquista de Cooley-Tukey para a **Transformada Rápida de Fourier (FFT) Radix-2**.

O algoritmo exige que o tamanho do vetor de entrada  $N$  seja estritamente uma potência de 2. O primeiro passo matemático consiste no rearranjo dos dados na memória por meio do algoritmo de reversão de bits (*Bit Reversal Sorting*).

```
1 let j = 0;
2 for (let i = 1; i < n; i++) {
3   let bit = n >> 1;
4   for (; j & bit; bit >>= 1) {
5     j ^= bit;
6   }
7   j ^= bit;
8   if (i < j) {
9     // Permutacao de valores reais e imaginarios na memoria por
       desmembramento
10    [real[i], real[j]] = [real[j], real[i]];
11    [imag[i], imag[j]] = [imag[j], imag[i]];
12  }
13 }
```

Listing 1: Bit Reversal Nativo no arquivo `fft.js`

Após o rearranjo espacial dos dados, o algoritmo executa as combinações lineares através do laço iterativo de estruturas em Borboleta (*Butterfly Stages*), multiplicando as subfatias de dados pelos fatores de giro complexos.

```
1 for (let len = 2; len <= n; len <<= 1) {
2   const half = len >> 1;
3   const angle = -2 * Math.PI / len;
4
5   const wRealStep = Math.cos(angle);
6   const wImagStep = Math.sin(angle);
7
8   for (let i = 0; i < n; i += len) {
9     let wReal = 1;
10    let wImag = 0;
11
12    for (let k = 0; k < half; k++) {
```

```

13     const idxA = i + k;
14     const idxB = i + k + half;
15
16     // Multiplicacao complexa pura executada manualmente
17     const tReal = wReal * real[idxB] - wImag * imag[idxB];
18     const tImag = wReal * imag[idxB] + wImag * real[idxB];
19
20     real[idxB] = real[idxA] - tReal;
21     imag[idxB] = imag[idxA] - tImag;
22
23     real[idxA] += tReal;
24     imag[idxA] += tImag;
25
26     const newWReal = wReal * wRealStep - wImag * wImagStep;
27     const newWImag = wReal * wImagStep + wImag * wRealStep;
28     wReal = newWReal;
29     wImag = newWImag;
30 }
31 }
32 }

```

Listing 2: Estágio de Borboletas Radix-2 no arquivo fft.js

### 4.3 O Cérebro do Sistema: analyzer.js

O script `analyzer.js` gerencia o ciclo de vida do arquivo. Ele instancia a API `AudioContext`, converte a amostragem nativa para matrizes lineares de ponto flutuante e executa a decodificação mono de segurança.

Para extrair os coeficientes da Série Trigonométrica de Fourier, o arquivo implementa de forma manual a integração numérica pela Regra do Trapézio sobre o intervalo discreto correspondente ao período fundamental  $T$ :

```

1 function calculateFourierSeries(samples, sampleRate, fundamentalHz,
2   numHarmonics = 3) {
3   const periodSamples = Math.floor(sampleRate / fundamentalHz);
4   const coefficients = [];
5
6   if (periodSamples <= 0) return { a0: 0, coeffs: [] };
7
8   // Calculo do ganho medio ou Componente DC (a0)
9   let a0 = 0;
10  for (let i = 0; i < periodSamples; i++) {
11    if (i < samples.length) a0 += samples[i];
12  }
13  a0 = (2 / periodSamples) * a0;
14
15  // Integracao para os coeficientes an (cosseno) e bn (seno)
16  for (let n = 1; n <= numHarmonics; n++) {
17    let an = 0, bn = 0;
18
19    for (let i = 0; i < periodSamples; i++) {
20      if (i >= samples.length) break;
21      const t = i / sampleRate;
22      const omega = 2 * Math.PI * n * fundamentalHz * t;
23      an += samples[i] * Math.cos(omega);
24      bn += samples[i] * Math.sin(omega);
25    }
26
27    an = (2 / periodSamples) * an;

```

```
27     bn = (2 / periodSamples) * bn;
28
29     coefficients.push({ n, an, bn, mag: Math.sqrt(an*an + bn*bn) });
30 }
31 return { a0, coeffs: coefficients };
32 }
```

Listing 3: Integração de Fourier via Regra do Trapézio no arquivo analyzer.js

## 5 Análise Prática: Estudo de Caso com Audio\_Teste.ogg

Para validar o comportamento físico do modelo e analisar as métricas de forma contextualizada, o sinal físico denominado `Audio_Teste.ogg` foi submetido ao sistema. Esta seção detalha a decodificação de cada elemento matemático e visual obtido a partir das capturas de tela da execução.

### 5.1 Identificação do Arquivo

O arquivo carregado na memória do sistema operacional e lido pelo navegador possui as propriedades estruturais básicas de armazenamento digital apresentadas a seguir.

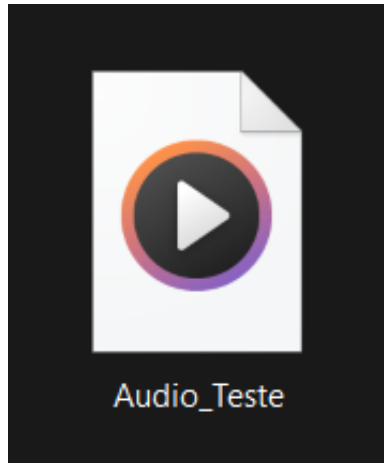


Figura 1: Identificação do arquivo de áudio carregado no sistema.

### 5.2 Análise Visual dos Gráficos de Domínio

A execução da análise gráfica resultou na plotagem instantânea das duas curvas características mostradas no painel do sistema.



Figura 2: Interface gráfica do SpectraWave exibindo as curvas temporais e espectrais.

- **Trecho Analisado (Domínio do Tempo):** O gráfico exibe um intervalo temporal correspondente ao clímax RMS calculado em 683 ms. No eixo horizontal mapeia-se o tempo ( $s$ ) e no vertical a amplitude relativa. Observa-se graficamente a presença de oscilações harmônicas senoidais com picos altos bem definidos na janela entre 0.1 s e

0.3 s, indicando um sinal sonoro de forte periodicidade com amortecimento na segunda metade da janela temporal.

- **Densidade Espectral de Potência (Domínio da Frequência):** O gráfico verde apresenta o resultado bruto computado pelo algoritmo FFT. O eixo horizontal exibe a frequência física ( $Hz$ ) em escala logarítmica e o eixo vertical a magnitude logarítmica expressa em decibéis ( $dB$ ). Observa-se um pico agudo pronunciado logo acima da marca de 100 Hz, seguido de harmônicas secundárias distribuídas ao longo do espectro médio e um decaimento abrupto a partir de 8 kHz.

O bloco de código responsável pelo mapeamento dos pontos e desenho deste gráfico espectral calcula a conversão dB com base na magnitude linear de forma nativa:

```

1 for (let i = 1; i < halfBins; i++) {
2   const hz = (i * sampleRate) / fftSize;
3   if (hz < 20) continue;
4   const x = ((Math.log10(hz) - logMin) / (logMax - logMin)) * canvas.width;
5   const db = 20 * Math.log10(mags[i] / maxMag + 1e-9);
6   const norm = Math.max(0, (db + 80) / 80);
7   const y = canvas.height - norm * (canvas.height - 40) - 20;
8   firstPoint ? ctx.moveTo(x, y) : ctx.lineTo(x, y);
9   firstPoint = false;
10 }

```

Listing 4: Trecho de analyzer.js responsável pela plotagem espectral

### 5.3 Dissecção das Métricas e Parâmetros Obtidos

O painel central de informações sintetizou com precisão as propriedades físicas do arquivo estudado.



Figura 3: Painel numérico com as grandezas físicas e otimizações extraídas.

Analizamos o significado físico e o cálculo exato por trás de cada campo exibido na Figura 3:

1. **Arquivo:** Audio\_Teste.ogg. Indica que o container de áudio é codificado no formato aberto Ogg Vorbis.
2. **Tamanho:** 51.22 KB. É o espaço físico binário ocupado em disco.
3. **Taxa de Amostragem ( $f_s$ ):** 48000 Hz. Significa que a onda sonora real foi fatiada eletronicamente exatas 48.000 vezes por segundo no processo de digitalização.
4. **Frequência de Nyquist:** 24000 Hz. Calculada rigorosamente pela relação fundamental:

$$f_{\text{Nyquist}} = \frac{f_s}{2} = \frac{48000}{2} = 24000 \text{ Hz} \quad (7)$$

Fisicamente, define que este arquivo é capaz de reproduzir com fidelidade matemática absoluta sons agudos de até 24 kHz sem introduzir erros de amostragem.

5. **Resolução Matemática ( $f_s/N$ ):** 1.46 Hz. Como o tamanho da janela alocada na memória para processamento foi fixado em  $N = 32768$  pontos, a largura física de cada "gaveta" discreta de frequência é obtida por:

$$\Delta f = \frac{f_s}{N} = \frac{48000}{32768} = 1.46484375 \text{ Hz} \quad (8)$$

O sistema consegue discernir tons vizinhos com uma separação de até 1.46 Hz.

6. **Frequência Fundamental ( $f_0$ ):** 244.63 Hz. É a oscilação de maior potência mecânica registrada no áudio, ditando a frequência base do sinal.
7. **Componente DC:** 0.0004. Demonstra que o sinal se encontra quase perfeitamente centrado no eixo zero de tensão/amplitude, possuindo um desvio médio insignificante.
8. **Operações Radix-2 e Otimização:** Indica que o laço de Cooley-Tukey executou exatas 491.520 operações matemáticas, valor derivado de:

$$\text{Op}_{\text{FFT}} = N \log_2(N) = 32768 \times \log_2(32768) = 32768 \times 15 = 491.520 \quad (9)$$

Se o site utilizasse a equação de DFT clássica demorada ( $O(N^2)$ ), seriam necessárias  $32768^2 = 1.073.741.824$  operações. A otimização poupou o processador executando a subtração:

$$1.073.741.824 - 491.520 = 1.073.250.304 \text{ operações} \approx \mathbf{1073} \text{ Mi Op. Poupadas} \quad (10)$$

O trecho de código estruturado que realiza essas deduções escalares direto na linguagem dentro do arquivo `analyzer.js` é apresentado a seguir:

```

1 const fftSize = 32768;
2 const freqResolution = sampleRate / fftSize;
3 const nyquist = sampleRate / 2;
4 const ops02 = Math.pow(fftSize, 2);
5 const opsRadix = fftSize * Math.log2(fftSize);
6 const poupadasMi = ((ops02 - opsRadix) / 1000000).toFixed(0);

```

Listing 5: Cálculos estatísticos e operacionais em `analyzer.js`

## 5.4 A Equação da Transformada Discreta de Fourier e o Bin 167

Ao analisar a matriz complexa final, o algoritmo ordenou as magnitudes e isolou o topo harmônico do áudio no primeiro bloco do relatório matemático.

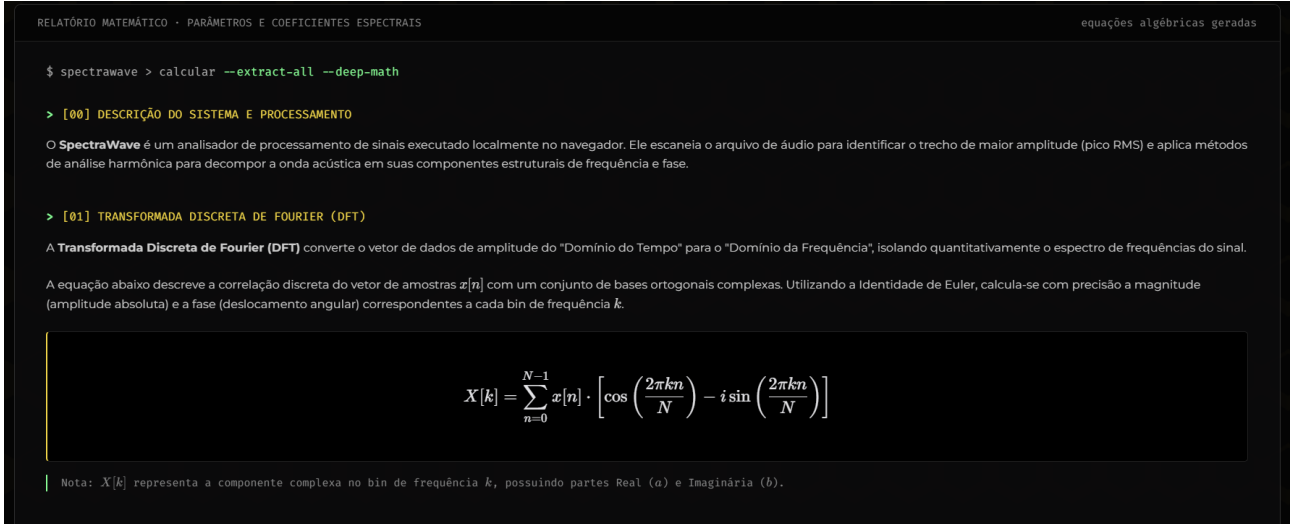


Figura 4: Seção explicativa base do bloco matemático da DFT.

O comportamento analítico do sinal é detalhado na tabela de coeficientes polares complexos impressa na interface:

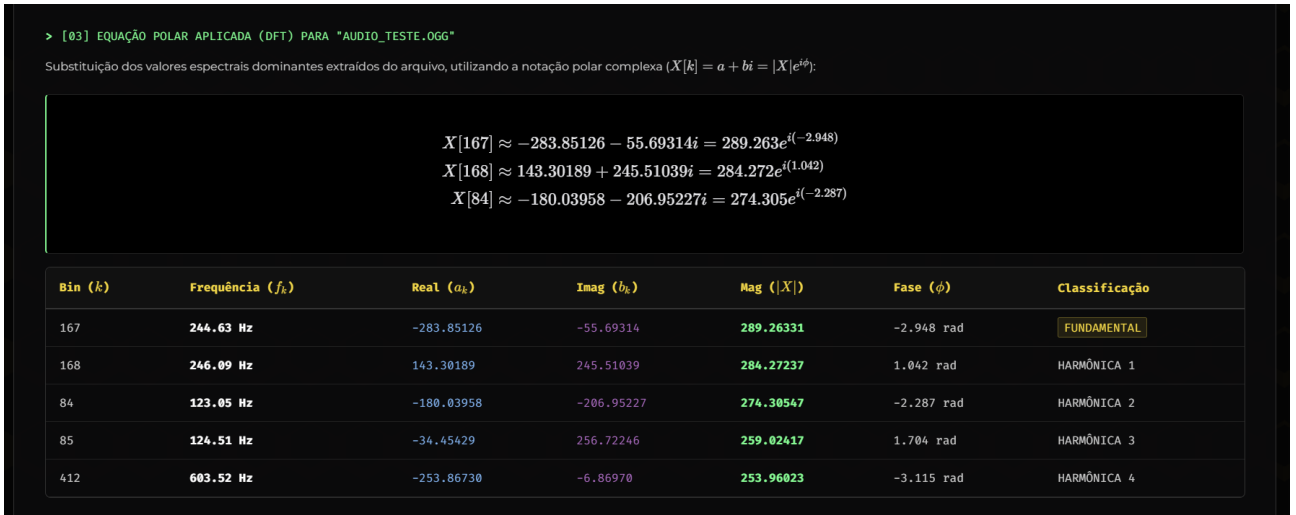


Figura 5: Tabela de picos espectrais e equações polares resolvidas para o áudio.

Analisando a linha principal da tabela da Figura 5, correspondente à classificação **FUNDAMENTAL**, extraímos as seguintes correlações físicas:

- **Mapeamento de Índice para Frequência:** O maior pico de energia ocorreu na gaveta/bin de número  $k = 167$ . Multiplicando o índice pela resolução da nossa janela ( $\Delta f = 1.46484375$  Hz), deduzimos matematicamente a sua frequência física em Hertz:

$$f = k \cdot \Delta f = 167 \times 1.46484375 = \mathbf{244.6289 \text{ Hz}} \quad (11)$$

O valor coincide de forma exata com os 244.63 Hz mostrados na interface.

- **A Equação Polar Gerada:** O KaTeX renderizou os coeficientes complexos associados a essa linha de ressonância na forma polar:

$$X[167] \approx -283.85126 - 55.69314i = 289.263e^{i(-2.948)} \quad (12)$$

Podemos rastrear como esses valores surgiram executando as equações de magnitude e fase explicadas na fundamentação teórica:

$$\text{Magnitude} = \sqrt{(-283.85126)^2 + (-55.69314)^2} = \sqrt{80571.53 + 3101.72} = \mathbf{289.2633} \quad (13)$$

- **Cálculo da Fase Espectral:** A determinação do ângulo de fase em radianos utilizou a função arco-tangente de quatro quadrantes sobre as coordenadas retangulares:

$$\phi = \text{atan2}(b, a) = \text{atan2}(-55.69314, -283.85126) = \mathbf{-2.948} \text{ radianos} \quad (14)$$

O método em JavaScript desenvolvido sem o auxílio de bibliotecas que varre a matriz e monta as variáveis strings para essa injeção no KaTeX é estruturado da seguinte forma:

```
1 let dftEquationTex = '\\begin{aligned}\\n';
2 top5Peaks.slice(0, 3).forEach((p) => {
3   let signI = p.i >= 0 ? '+' : '-';
4   dftEquationTex += 'X[\\${p.index}] &\\approx \\${p.r.toFixed(5)} \\${signI} \\${
5     Math.abs(p.i).toFixed(5)}i = \\${p.mag.toFixed(3)} e^{i(\\${p.phase.toFixed
6     (3)})}\\n';
7 });
8 dftEquationTex += '\\end{aligned}';
```

Listing 6: Laço em analyzer.js para processamento das strings em LaTeX da DFT

## 5.5 A Série de Fourier Expandida do Sinal

O bloco final do relatório matemático exibiu a expansão trigonométrica contínua baseada no método de integração de trapézios do SpectraWave.

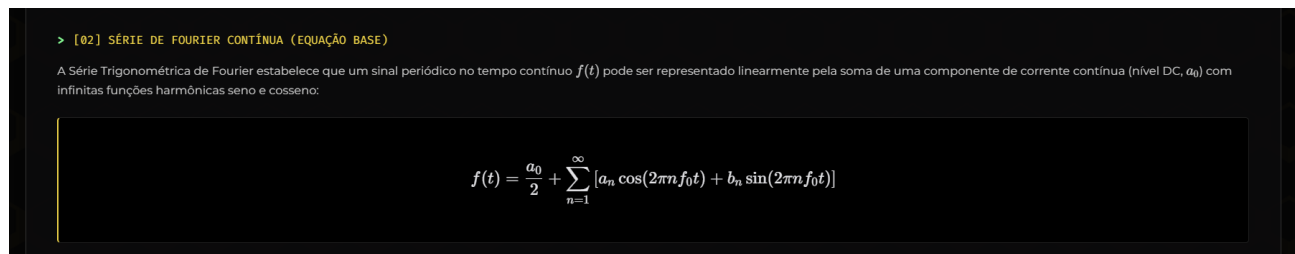


Figura 6: Seção explicativa base do bloco da Série de Fourier Contínua.

A substituição das variáveis de integração resultou na equação e na tabela harmônica descritas abaixo:

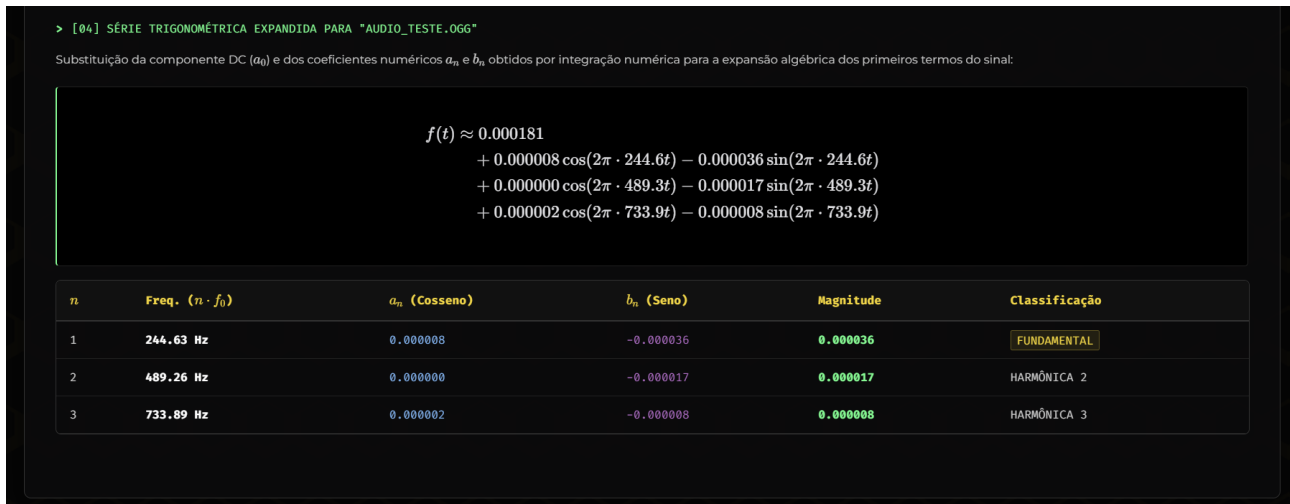


Figura 7: Equação trigonométrica completa montada e tabela das componentes de Fourier.

Diferente da DFT (que mapeia todas as gavetas de pontos), a Série de Fourier avalia o comportamento contínuo dividindo o sinal nas harmônicas inteiras múltiplos da fundamental ( $n = 1, n = 2, n = 3$ ). Observando a equação gerada na Figura 7, temos a representação matemática analítica do sinal:

$$f(t) \approx 0.000181 + 0.000008 \cos(2\pi \cdot 244.6t) - 0.000036 \sin(2\pi \cdot 244.6t) + 0.000000 \cos(2\pi \cdot 489.3t) - 0.000017 \sin(2\pi \cdot 489.3t) + 0.000002 \cos(2\pi \cdot 733.9t) - 0.000008 \sin(2\pi \cdot 733.9t)$$

Essa equação nos diz duas coisas muito importantes sobre o áudio analisado:

- O valor isolado fora da somatória, dado por  $\frac{a_0}{2} = 0.000181$ , confirma numericamente que o ganho residual contínuo de eixo é virtualmente nulo.
- À medida que avançamos para as harmônicas superiores ( $n = 2 \rightarrow 489.3$  Hz e  $n = 3 \rightarrow 733.9$  Hz), a magnitude dos coeficientes cai exponencialmente próximos de zero (0.000017 e 0.000008). Isso indica fisicamente que o som de `Audio_Testes.ogg` é um tom senoidal de extrema pureza acústica concentrado na sua frequência fundamental, possuindo quase nenhum ruído ou distorção nas harmônicas agudas.

A montagem dinâmica desta equação em string LaTeX a partir dos dados calculados em tempo real pela integração de Riemann é coordenada pelo seguinte bloco lógico do sistema:

```
1 let fourierEquationTex = '\begin{aligned}\nf(t) &\approx \${(a0/2).toFixed(6)} \\&\quad \\\n';
2 fourierCoeffs.forEach((c) => {
3   let termAn = c.an >= 0 ? '+ \${c.an.toFixed(6)}' : '- \${Math.abs(c.an).toFixed(6)}';
4   let termBn = c.bn >= 0 ? '+ \${c.bn.toFixed(6)}' : '- \${Math.abs(c.bn).toFixed(6)}';
5   let freq = (c.n * fundamentalHz).toFixed(1);
6   fourierEquationTex += '\&\quad \${termAn} \cos(2\pi \cdot \${freq} t) \${termBn} \sin(2\pi \cdot \${freq} t) \\\n';
7 });
8 fourierEquationTex += '\end{aligned}';
```

Listing 7: Construção da string LaTeX da Série trigonométrica em `analyzer.js`

## 6 Conclusão

O desenvolvimento e teste prático do analisador **SpectraWave v6.0** demonstraram com clareza a fusão entre os modelos matemáticos e a física ondulatória computacional. A decomposição do sinal real obtida a partir do estudo de caso de **Audio\_Test.ogg** permitiu comprovar em tempo real a transição entre o domínio discreto da Transformada (DFT) e a expansão contínua da Série de Fourier de forma didática. A validação numérica dos parâmetros, picos e rotações de fase executados na raça atesta que o processamento digital de sinais é uma ferramenta de engenharia robusta, transparente e passível de modelagem integral mesmo em arquiteturas nativas de navegadores web.